# Algorithm for file updates in Python

## Project description

In this project, I created a Python algorithm to manage access to restricted content. My company uses an "allow list" of IP addresses to control who can see sensitive patient data. Sometimes, employees no longer need access, so their IP addresses must be removed from the list.

Doing this task manually takes a long time and can lead to mistakes. I used Python to automate the process: reading the file, filtering out old IP addresses, and updating the list. This makes the security process faster and more reliable.

## Open the file that contains the allow list

To start, I used the `open()` function with the `"r"` (read) parameter to access the "allow_list.txt" file. I used the `with` statement for this process. The `with` statement is important in cybersecurity because it automatically closes the file after the task is finished. This helps manage computer resources and prevents errors.

## Read the file contents

After opening the file, I used the `.read()` method to convert the file's content into a string. I stored this string in a variable named `ip_addresses`. This allowed me to see the list of authorized IP addresses as a single piece of text that I could later process with Python.

```python
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"
# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]
# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:
  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
  ip_addresses = file.read()
# Display `ip_addresses`
print(ip_addresses)
```

```
192.168.25.60 192.168.6.9 192.168.90.124 192.168.133.188 192.168.218.219 192.168.156.224 192.168.69.116
```

## Convert the string into a list

The data I read from the file is currently a single string. To process the IP addresses individually, I need to convert this string into a list. I used the `.split()` method for this task. By default, `.split()` separates the text at each whitespace, creating a list where each element is a single IP address. This makes it much easier to iterate through the list and remove specific addresses later.

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"
# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]
# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:
  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
  ip_addresses = file.read()
# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()
# Display `ip_addresses`
print(ip_addresses)
```

```
['192.168.25.60', '192.168.6.9', '192.168.90.124', '192.168.133.188', '192.168.218.219', '192.168.156.224', '192.168.69.116']
```

## Iterate through the remove list

To update the list, I used a `for` loop. This loop allowed the algorithm to look at every IP address in the `ip_addresses` list one by one. By iterating through the list, the program can check each address individually to see if it still belongs in the authorized group.

## Remove IP addresses that are on the remove list

Inside the loop, I added a conditional `if` statement. This statement checks if the current IP address is also present in the `remove_list`. If an address is found in both lists, I used the `.remove()` method to delete it from the `ip_addresses` list. This ensures that any user who

should no longer have access is successfully filtered out.

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Display `ip_addresses`

print(ip_addresses)
```

```
['192.168.25.60', '192.168.6.9', '192.168.90.124', '192.168.133.188', '192.168.218.219', '192.168.156.224', '192.16
8.69.116']
```

## Update the file with the revised list of IP addresses

To finalize the update, I first converted the `ip_addresses` list back into a string using the `.join()` method. I used a space `" "` as a separator between each address. Then, I opened the "allow_list.txt" file again, but this time I used the `"w"` (write) parameter. This allowed me to

overwrite the old content with the new, updated list of authorized IP addresses.

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access
#restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file

ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file

with open(import_file, "w") as file:

  # Rewrite the file, replacing its contents with `ip_addresses`

  file.write(ip_addresses)

# Build `with` statement to read in the updated file

with open(import_file, "r") as file:

    # Read in the updated file and store the contents in `text`

    text = file.read()

# Display the contents of `text`

print(text)
```

```
192.168.25.60 192.168.6.9 192.168.90.124 192.168.133.188 192.168.218.219 192.168.156.224 19
2.168.69.116
```

## Summary

In summary, I developed a Python algorithm that automates the management of an IP allow list. The script successfully reads a text file, converts the data into a list, and uses a loop to remove unauthorized IP addresses. Finally, it saves the updated list back to the file. This automation reduces manual work and improves the security of sensitive data by ensuring only authorized users have access. This project demonstrates my ability to use Python for practical cybersecurity tasks like access control and data filtering.

```python
# Define a function named `update_file` that takes in two parameters: `import_file` and
#`remove_list`
# and combines the steps you've written in this lab leading up to this

def update_file(import_file, remove_list):

  # Build `with` statement to read in the initial contents of the file

  with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

    ip_addresses = file.read()

  # Use `.split()` to convert `ip_addresses` from a string to a list

  ip_addresses = ip_addresses.split()

  # Build iterative statement
  # Name loop variable `element`
  # Loop through `ip_addresses`

  for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in remove_list:

      # then current element should be removed from `ip_addresses`

      ip_addresses.remove(element)

  # Convert `ip_addresses` back to a string so that it can be written into the text file

  ip_addresses = " ".join(ip_addresses)

  # Build `with` statement to rewrite the original file

  with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`

    file.write(ip_addresses)

# Call `update_file()` and pass in "allow_list.txt" and a list of IP addresses to be removed

update_file("allow_list.txt", ["192.168.25.60", "192.168.140.81", "192.168.203.198"])

# Build `with` statement to read in the updated file

with open("allow_list.txt", "r") as file:

  # Read in the updated file and store the contents in `text`

  text = file.read()

# Display the contents of `text`

print(text)
```

```
192.168.6.9 192.168.90.124 192.168.133.188 192.168.218.219 192.168.156.224 192.168.69.116
```